Definition of SDLC

SDLC or the Software Development Life Cycle is a process that produces software with the highest quality and lowest cost in the shortest time. SDLC includes a detailed plan for how to develop, alter, maintain, and replace a software system.

SDLC involves several distinct stages, including planning, design, building, testing, and deployment. Popular SDLC models include the **waterfall model**, **spiral model**, **and Agile model.**

How SDLC Works

SDLC works by lowering the cost of software development while simultaneously improving quality and shortening production time. SDLC achieves these apparently divergent goals by following a plan that removes the typical pitfalls to software development projects. That plan starts by evaluating existing systems for deficiencies. Next, it defines the requirements of the new system. It then creates the software through the stages of design, development, testing, and deployment. By anticipating costly mistakes like failing to ask the end user for suggestions, SLDC can eliminate redundant rework and after-the-fact fixes.

Stages and Best Practices of SDLC

Following the best practices and/or stages of SDLC ensures the process works in a smooth, efficient, and productive way.

1. **Identify the** current problems**.** "What don't we want?" This stage of SDLC means getting input from all stakeholders, including customers, salespeople, industry experts, and programmers. Learn the strengths and weaknesses of the current system with improvement as the goal.
2. **Plan.** "What do we want?" In this stage of SDLC, the team

defines the requirements of the new software and determines the cost and resources required. It also details the risks involved and provides sub-plans for softening those risks. In this stage, a Software Requirement Specification document is created.

3. **Design.** “How will we get what we want?” This phase of SDLC starts by turning the software specifications into a design plan called the Design Specification. All stakeholders then review this plan and offer feedback and suggestions. It’s crucial to have a plan for collecting and incorporating stakeholder input into this document. Failure at this stage will almost certainly result in cost overruns at best and total collapse of the project at worst.
4. **Build.** “Let’s create what we want.” This SDLC stage develops the software by generating all the actual code. If the previous steps have been followed with attention to detail, this is actually the least complicated step.
5. **Test.** “Did we get what we want?” In this stage, we test for defects and deficiencies. We fix those issues until the product meets the original specifications.
6. **Deploy.** “Let’s start using what we got.” Often, this part of the SDLC process happens in a limited way at first. Depending on feedback from end users, more adjustments can be made.
7. **Maintain.** “Let’s get this closer to what we want.” The plan almost never turns out perfect when it meets reality. Further, as conditions in the real world change, we need to update and advance the software to match.

Examples of SDLC in Action

The most common SDLC examples or SDLC models are listed below.

- **Waterfall Model.** This SDLC model is the oldest and most straightforward. With this methodology, we finish one phase

and then start the next. Each phase has its own mini-plan and each phase “waterfalls” into the next. The biggest drawback of this model is that small details left incomplete can hold up the entire process.

- **Agile Model.** The Agile SDLC model separates the product into cycles and delivers a working product very quickly. This methodology produces a succession of releases. Testing of each release feeds back info that’s incorporated into the next version. According to Robert Half, the drawback of this model is that the heavy emphasis on customer interaction can lead the project in the wrong direction in some cases.
- **Iterative Model.** This SDLC model emphasizes repetition. Developers create a version very quickly and for relatively little cost, then test and improve it through rapid and successive versions. One big disadvantage here is that it can eat up resources fast if left unchecked.
- **V-Shaped Model.** An extension of the waterfall model, this SDLC methodology tests at each stage of development. As with waterfall, this process can run into roadblocks.
- **Big Bang Model.** This high-risk SDLC model throws most of its resources at development and works best for small projects. It lacks the thorough requirements definition stage of the other methods.
- **Spiral Model.** The most flexible of the SDLC models, the spiral model is similar to the iterative model in its emphasis on repetition. The spiral model goes through the planning, design, build and test phases over and over, with gradual improvements at each pass.

Benefits of SDLC

SDLC done right can allow the highest level of management control and documentation. Developers understand what they should build and why.

All parties agree on the goal up front and see a clear plan for arriving at that goal. Everyone understands the costs and resources required.

Several pitfalls can turn an SDLC implementation into more of a roadblock to development than a tool that helps us. Failure to take into account the needs of customers and all users and stakeholders can result in a poor understanding of the system requirements at the outset. The benefits of SDLC only exist if the plan is followed faithfully.

System design is divided into two types:

Logical Design : The logical design of a system pertains to an abstract representation of the data flows, inputsand outputs of the system. This is often conducted via modeling, which involves a simplistic(and sometimes graphical) representation of an actual system. In the context of systems design, modeling can undertake the following forms, including Dataflow diagrams Entity Life Histories, Entity Relationship Diagram.

Physical Design : The physical design relates to the actual input and output processes of the system. This is laid down in terms of how data is inputted into a system, how it is verified/authenticated, how it is processed, and how it is displayed as output.

System Design includes following points:

i. Requirements analysis - analyzes the needs of the end users or customers

ii. Benchmarking — is an effort to evaluate how current systems are used

iii. Systems architecture - creates a blueprint for the design with the necessary specifications for the hardware, software, people and data resources. In many cases, multiple architectures are evaluated before one is selected.

iv. Design — designers will produce one or more 'models' of what they see a system eventually looking like, with ideas from the analysis section either used or discarded.A document will be produced with a description of the system, but nothing is specific— they might say 'touch screen' or 'GUI operating system', but not mention anyspecific brands.

v. Computer programming and debugging in the software world, or detailed design in the consumer, enterprise or commercial world - specifies the final system components.

vi. System testing - evaluates the system's actual functionality in relation to

expected orintended functionality, including all integration aspects.

Feasibility Study

Sometimes a feasibility study is done as part of a systems development lifecycle, in order to drive precision for the implementation of technologies. Engineers might look at a five-point model called TELOS — this includes the following components:

- Technical
- Economic
- Legal
- Operational
- Schedule

Under technical, engineers ask whether the correct technology exists to support a project. Under economic, they look at costs and benefits. Under legal, they look at any barriers to legal implementation, for instance, privacy issues or safety concerns. Under operational, they look at how systems can be maintained after being built. In schedule, they look at chronology for a project.

In general, a feasibility study should cover all of the salient points on whether a project is reasonable, and should have documentation related to any concerns.

JAD APPROACH

JAD (Joint Application Development) is a software development approach which engages the client and/or the end users for designing and developing the system. This model was designed and put forward by Dr. Chuck Morris and Dr. Tony Crawford of IBM, who propose this model in the late 1970s. As compared to other primitive SDLC model, Joint Application Development model leads to faster progression of the system development which has better client approval.
This model furthermore, is vast when it comes to agile delivery wherein the software products need to be developed as well as shipped in short iterations depending on agreements among the industrial as well as industry stakeholders which are termed as Minimum Viable Product (MVP)

Phases of JAD model

Since you have become familiar with the JAD concept, it is time to know about its phases and how the model's design and development approach works:

1. Define Specific Objectives: The facilitator, in partnership with stakeholders, set all the objectives as well as a list of items which is then distributed to other developers and participants to understand and review. This objective contains elements like the scope of this projected system, its potential outcome, technical specification required, etc.
2. Session Preparation: The facilitator is solely responsible for this preparation where all relevant data is collected and sent to other members before time. For better insight, research carried out to know about the system requirement better and gather all the necessary information for development.
3. Session Conduct: Here the facilitator is accountable to identify those issues which have to be working out for making the system error-free. Here the facilitator will serve as a participant but will not have a say regarding any information.
4. Documentation: After the product is developed, the records and published documents are put forward into the meeting so that the stakeholders and consumers can approve it through the meeting

Benefits of using JAD MODEL

- Improved Delivery Time: The time required for developing a product using JAD model is lesser and efficient than that of other traditional models.
- Cost Reduction: Efficiently analyzing the requirements and facts with business executives and stakeholders will make less effort to develop the system and hence less cost will be required for the entire development process.
- Better Understanding: Since the entire requirement is analyzed by business executives, followed by a cautious choice of developers and team member who can professionally interact with each other better usually helps in understanding the product development better.
- Improved Quality: Since all the key decision makers and stakeholders of the project are involved in the development of the project so there is the least chance of error and hence the product quality becomes better and more accurate.

Code walkthrough is informal process where code is executed at any time. Author of code leads the code walkthrough code review is formal process where review is done line by line in the formal meeting of developers and QA engineers.

Code review : A meeting at which software code is presented to project personnel, managers, users, customers, or other interested parties for comment or approval.It is systematic examination (sometimes referred to as peer review) of computer source code. It is intended to find mistakes overlooked in the initial development phase, improving the overall quality of software. Reviews are

done in various forms such as pair programming, informal walkthroughs, and formal inspections.

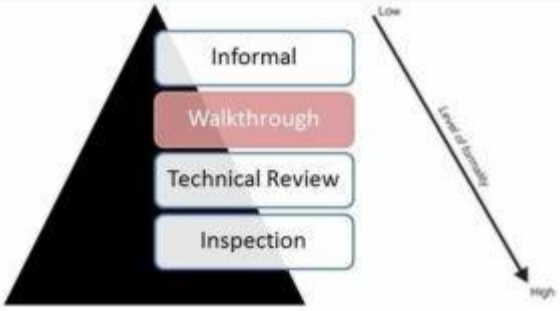
Code review may be especially productive for identifying security vulnerabilities. Specialized application programs are available that can help with this process. Automated code reviewing facilitates systematic testing of source code for potential trouble such as buffer overflows, race conditions, memory leakage, size violations, and duplicate statements. Code review is also commonly done to test the quality of patches.

Inspection

An inspection is, most generally, an organized examination or formal evaluation exercise. It involves the measurements, tests, and gauges applied to certain characteristics in regard to an object or activity. The results are usually compared to specified requirements and standards for determining whether the item or activity is in line with these targets. Inspections are usually non-destructive.

Code Walkthrough – walk-through is a form of software peer review "in which a designer or programmer leads members of the development team and other interested parties through a software product, and the participants ask questions and make comments about possible errors, violation of development standards, and other problems

A walkthrough is a term describing the consideration of a process at an abstract level.



code_walkthrough

The term is often employed in the software industry (see software walkthrough) to describe the process of inspecting algorithms and source code by following paths through the algorithms or code as determined by input conditions and choices made along the way. The purpose of such code walkthroughs is generally to provide assurance of the fitness for purpose of the algorithm or code; and occasionally to assess the competence or output of an individual or team.

The term is employed in the theatrical and entertainment industry to describe a rehearsal where the major issues of choreography and interaction are practiced and resolved, prior to more formal "dress rehearsals".

The term is often used in the world of learning where a tutor/trainer will walk through the process

for the first time. It is regarded as a literal walk through of the learning at the groups pace ensuring that everyone takes in the new knowledge and skills.
Something akin to walkthroughs are used in very many forms of human endeavour since the process is a thought experiment that seeks to determine the likely outcome(s) of an affair based on starting conditions and the effects of decisions taken.

What is Software Quality Metrics?

The word '**metrics**' refer to standards for measurements. Software Quality Metrics means measurement of attributes, pertaining to software quality along with its process of development.

The term **"software quality metrics"** illustrate the picture of measuring the software qualities by recording the number of defects or security loopholes present in the software. However, quality measurement is not restricted to counting of defects or vulnerabilities but also covers other aspects of the qualities such as maintainability, reliability, integrity, usability, customer satisfaction, etc.

Why Software Quality Metrics?

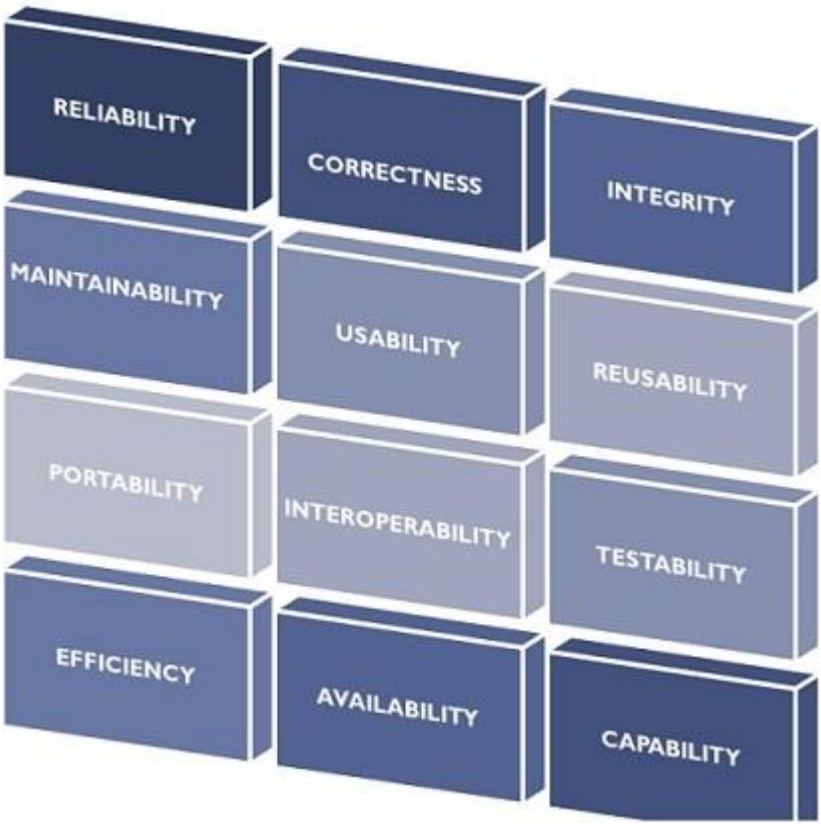
1. To define and categorize elements in order to have better understanding of each and every process and attribute.

2. To evaluate and assess each of these process and attribute against the given requirements and specifications.

3. Predicting and planning the next move w.r.t software and business requirements.

4. Improving the Overall quality of the process and product, and subsequently of project.

<u>What are the quality factors that define & impact the user experience?</u>



Software Quality Metrics: sub-category of Software Metrics

It is basically, a subclass of **software metrics** that mainly emphasizes on quality assets of the software product, process and project. Software metric is a broader concept that incorporates software quality metrics in it, and mainly consists of three types of metrics:

- **Product Metrics:**

  It includes size, design, complexity, performance and other parameters that are associated with the product's quality.

- **Process Metrics:**

  It involves parameters like, time-duration in locating and removing defects, response time for resolving issues, etc.

- **Project Metrics:**

  It may include number of teams, developers involved, cost and duration for the project, etc.

Features of good Software Quality Metrics

- Should be specific to measure the particular attribute or an attribute of greater importance.

- Comprehensive for wide variety of scenarios.

- Should not consider attributes that have already been measured by some other metric.
- Reliable to work similarly in all conditions.
- Should be easy and simple to understand and operate